COMPUTER VISION

LAB SHEET :06

```
# Name: Deepti
# CUID:24250168
# Roll No.: 16   Sec:B

import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# 1. LOAD THE IMAGE
# Keep image.jpg in the same folder as this Python file
image = cv2.imread("image.jpg")
if image is None:
    print("Error: Image could not be loaded.")
    print("Make sure image.jpg is in the same folder as this Python file.")
    exit()
else:
    print("Image loaded successfully.")
    print("Image size:", image.shape)

# 2. CONVERT IMAGE TO GRAYSCALE
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.figure(figsize=(6, 5))
plt.imshow(gray, cmap="gray")
```

```
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

# 3. APPLY GAUSSIAN BLUR
blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)
plt.figure(figsize=(6, 5))
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")
plt.show()

# 4. GLOBAL THRESHOLDING
threshold_value = 127

_, global_threshold = cv2.threshold(
    blurred,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(6, 5))
plt.imshow(global_threshold, cmap="gray")
```

```
plt.title("Global Thresholding")
plt.axis("off")
plt.show()

# 5. OTSU'S THRESHOLDING
otsu_threshold_value, otsu_threshold = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("\n========== OTSU THRESHOLD ==========")
print("Optimal Otsu Threshold Value:", otsu_threshold_value)

plt.figure(figsize=(6, 5))
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()

# 6. ADAPTIVE THRESHOLDING

adaptive_threshold = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
```

```
    11,
    2
)

plt.figure(figsize=(6, 5))
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()

# 7. WATERSHED SEGMENTATION

# Convert image to binary using Otsu's threshold
_, binary = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove noise using morphological opening
kernel = np.ones((3, 3), np.uint8)

opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
```

```
)

# Find sure background
sure_background = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Find distance transform
distance = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Find sure foreground
_, sure_foreground = cv2.threshold(
    distance,
    0.7 * distance.max(),
    255,
    0
)

sure_foreground = np.uint8(sure_foreground)

# Find unknown region
unknown = cv2.subtract(
```

```
    sure_background,
    sure_foreground
)

# Create markers
num_markers, markers = cv2.connectedComponents(
    sure_foreground
)

markers = markers + 1

markers[unknown == 255] = 0

# Apply watershed
watershed_markers = cv2.watershed(
    image.copy(),
    markers
)

watershed_result = image.copy()

# Mark boundaries in red
watershed_result[watershed_markers == -1] = [255, 0, 0]

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(watershed_result, cv2.COLOR_BGR2RGB))
plt.title("Watershed Segmentation")
plt.axis("off")
```

```
plt.show()

# 8. K-MEANS COLOR SEGMENTATION

# Convert image from BGR to RGB
rgb_image = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)

# Reshape image into a list of pixels
pixels = rgb_image.reshape(
    (-1, 3)
)

pixels = np.float32(pixels)

# Number of clusters
k = 3

# Define K-Means criteria
criteria = (
    cv2.TERM_CRITERIA_EPS +
    cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)
```

```
# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixels)

centers = np.uint8(
    kmeans.cluster_centers_
)

# Replace each pixel with its cluster center
segmented_pixels = centers[labels]

# Reshape back into image
kmeans_image = segmented_pixels.reshape(
    rgb_image.shape
)

plt.figure(figsize=(8, 6))
plt.imshow(kmeans_image)
plt.title("K-Means Segmentation")
plt.axis("off")
plt.show()

# 9. COMPARE SEGMENTATION TECHNIQUES
```

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(watershed_result, cv2.COLOR_BGR2RGB))
plt.title("Watershed")
plt.axis("off")
plt.subplot(2, 3, 6)
plt.imshow(kmeans_image)
plt.title("K-Means")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()

# 10. SAVE SEGMENTED IMAGES

cv2.imwrite(
    "global_threshold.jpg",
    global_threshold
)

cv2.imwrite(
    "otsu_threshold.jpg",
    otsu_threshold
)

cv2.imwrite(
    "adaptive_threshold.jpg",
    adaptive_threshold
)
cv2.imwrite(
    "watershed_result.jpg",
    watershed_result
)
cv2.imwrite(
    "kmeans_segmentation.jpg",
    cv2.cvtColor(
```

```
        kmeans_image,
        cv2.COLOR_RGB2BGR
    )
)
print("\nAll segmented images saved successfully!")

# OBSERVATIONS
# Global Thresholding:
# Global thresholding uses a fixed threshold value to separate the foreground
# from the background.
# It works well when illumination is relatively uniform.

# Otsu's Thresholding:
# Otsu's method automatically determines an optimal threshold value.
# It is useful when the image contains distinguishable foreground and background regions.

# Adaptive Thresholding:
# Adaptive thresholding calculates different threshold values for different local regions.
# It is useful for images with uneven illumination.

# Watershed Segmentation:
# Watershed segmentation treats the image as a topographic surface.
# It is useful for separating touching or overlapping objects.

# K-Means Segmentation:
# K-Means groups pixels into different clusters based on their color or intensity values.
# It is useful for color-based image segmentation.
# Comparison:
```

Thresholding methods are simple and effective for separating foreground and background.

Watershed is useful for separating touching objects.

K-Means is useful for color-based segmentation.

QUESTIONS AND ANSWERS

1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Answer: Image segmentation is the process of dividing an image into meaningful regions or objects.

It helps separate important objects or regions from the background.

Segmentation simplifies image analysis and supports tasks such as object detection,

medical diagnosis, autonomous navigation, and industrial inspection.

2. Differentiate between Image Segmentation and Image Classification.

Answer: Image segmentation divides an image into different regions or objects at the pixel level.

Image classification assigns a label or category to an entire image or object.

Segmentation provides information about the location and boundaries of objects,

while classification mainly identifies what the image or object represents.

3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Answer: Global Thresholding uses a fixed threshold value to divide pixels into foreground and background.

Otsu's Thresholding automatically selects an optimal threshold based on the image histogram.

Adaptive Thresholding calculates threshold values for different local regions of an image.

Therefore, adaptive thresholding is useful when illumination varies across the image.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Answer: The Watershed Algorithm is a segmentation method that treats an image as a topographic surface.

It identifies different regions based on local intensity and boundary information.

It is particularly useful for separating objects that are touching or overlapping.

Markers can be used to identify different objects and guide the segmentation process.

5. How is K-Means Clustering applied to image segmentation?

Answer: K-Means treats image pixels as data points and groups them into a specified number

of clusters based on their color or intensity values.

Each pixel is assigned to the nearest cluster center.

The cluster centers are repeatedly updated until the clustering becomes stable.

The resulting clusters represent different segmented regions of the image.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Answer: Threshold-based segmentation separates pixels using one or more threshold values.

It is simple and computationally efficient but can be affected by illumination changes.

Clustering-based segmentation groups pixels according to their similarity in features

such as color or intensity.

K-Means is an example of clustering-based segmentation and can be useful for color images.

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Answer: Complex backgrounds can contain objects and colors that are similar to the foreground.

Varying illumination can make a single threshold unsuitable for the entire image.

Noise, shadows, overlapping objects, and low contrast can also make segmentation difficult.

Adaptive thresholding, preprocessing, watershed, or clustering methods can help address these challenges.

8. Mention five real-world applications where image segmentation plays a critical role.

Answer: Image segmentation is used in medical imaging, autonomous navigation,

object detection, industrial inspection, and satellite image analysis.

In medical imaging, segmentation helps identify organs and abnormal regions.

In autonomous systems, it helps identify roads, objects, and obstacles.

It also helps isolate important regions in industrial and satellite images.

9. How does image segmentation improve the performance of object detection and image recognition systems?

Answer: Segmentation separates important objects or regions from the background.

This provides clearer object boundaries and reduces irrelevant background information.

Object detection and recognition algorithms can then focus on meaningful regions.

Therefore, segmentation can improve the accuracy and efficiency of subsequent computer vision tasks.

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Answer: Traditional segmentation techniques such as thresholding, watershed,

and K-Means are generally simple and require less computational resources.

However, their performance can decrease with complex images and varying conditions.

Deep learning methods such as U-Net and Mask R-CNN can learn complex features

and provide accurate segmentation for difficult images.

However, they generally require training data, greater computational resources,

and more complex implementation.

OUTPUT

Grayscale Image



Gaussian Blurred Image



Original



Global Thresholding



Otsu Thresholding



Adaptive Thresholding



Watershed



K-Means

